

NAVIS: Concurrent Search and Update with Low Position-Seeking Overhead in On-SSD Graph-Based Vector Search

Jaeyong Song, Hongsun Jang,
Changmin Shin,
Seongyeon Park, Yong Jae Ryoo
Seoul National University
Seoul, South Korea
{jaeyong.song,hongsun.jang}@snu.ac.kr
{scm8432,syeonp,jae8259}@snu.ac.kr

Seo Jin Park
University of Southern California
Los Angeles, California, USA
seojinpa@usc.edu

Jinho Lee
Seoul National University
Seoul, South Korea
leejinho@snu.ac.kr

ABSTRACT

On-disk graph-based vector search (GVS) has become the dominant approach for serving large-scale vector databases at high recall, but prior systems struggle to sustain concurrent search and update throughput on high-dimensional workloads. We find the main cause of this in *position seeking*, a full graph traversal that every update performs to locate neighbors before linking the new vector into the graph. Position seeking is fundamentally heavier than a search query, and its cost is further amplified by two systemic limitations of current GVS systems, packed layouts that couple every edge fetch to a full vector load, and a static entrance graph whose entry points drift away from newly inserted regions as updates accumulate. We present NAVIS, an on-SSD GVS system that drives down position-seeking overhead through (i) a layout-supported selective vector read that breaks the packed-page coupling without losing its locality benefits, (ii) a dynamic lightweight entrance graph update mechanism that reuses traversal information already produced by concurrent updates, and (iii) an entrance graph-aware edgelist cache that concentrates capacity on high-reuse paths near refreshed entry points. Across multiple large-scale high-dimensional benchmarks, NAVIS enhances average insertion throughput by up to 2.74 $\times$ and average concurrent search throughput by up to 1.37 $\times$ while reducing average search latency by up to 25.26%.

PVLDB Reference Format:

Jaeyong Song, Hongsun Jang, Changmin Shin,
Seongyeon Park, Yong Jae Ryoo, Seo Jin Park, and Jinho Lee. NAVIS:
Concurrent Search and Update with Low Position-Seeking Overhead in
On-SSD Graph-Based Vector Search. PVLDB, 20(X): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Modern applications like retrieval-augmented generation [33], recommendation systems [25, 35], and multimedia search [24, 63] increasingly rely on large-scale vector databases to retrieve semantically similar items at high recall. Among various designs, on-disk

graph-based vector search (GVS) [19, 20, 45, 49, 50, 54] has emerged as the dominant approach for high-recall search at scale. In on-disk GVS, vectors are vertices in a proximity graph whose edges link nearby points. Queries are answered via greedy traversal that hops from a starting vertex toward progressively closer neighbors, loading only the visited portions of the graph index from SSD on demand. Recent production deployments [11, 45] increasingly adopt on-disk GVS because it sustains high recall without holding full vectors in DRAM [17, 34], especially for high-dimensional vectors from encoders such as CLIP [47], OpenAI’s embedding models [42, 43], and BGE-M3 [5], with dimensionalities ranging from 512 to 3072.

With the rise of agentic systems [22, 29], and as production pipelines [10, 13, 17] now ingest fresh user activity and frequent uploads, the on-disk GVS systems are no longer static snapshots. There are two methods to handle such vector updates, *index rebuild* and *index update*. Since index rebuild often takes several days for large-scale datasets [50], it often fails to provide up-to-date search results. Thus, many approaches [20, 49, 55, 57] focus on index update to quickly incorporate fresh vectors into databases. However, sustaining high search performance under such concurrent index updates is non-trivial.

Prior index update methods in on-disk GVS systems take two main approaches. One approach [49, 57] temporarily buffers new updates in memory and merges them into the on-disk graph in bulk, which amortizes per-insertion I/O across many entries. However, this batched merge causes severe fluctuations in search performance during each merging window [20, 55]. To stabilize search performance while concurrently absorbing insertions, a second line of work [20, 55] adopts in-place updates that commit every update directly on disk as it arrives, avoiding the search latency spikes.

Although both lines of work make meaningful progress on update support in on-disk GVS, neither sustains the high concurrent search and update throughput that modern high-dimensional workloads demand. We find the main cause of this in *position seeking*, a graph traversal that every update performs to locate neighbors before linking the new vector into the graph.

Compared to a typical search, position seeking for updates is fundamentally heavier and demands far more I/O. A search query terminates once it collects a handful of adjacent neighbors because it only needs the top- K (e.g., 10) similar vectors. By contrast, an update must match the new vertex’s degree to the graph’s maximum out-degree (e.g., 64–96) to preserve graph quality, which incurs

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. X ISSN 2150-8097.
doi:XX.XX/XXX.XX

sizeable vector and neighbor list loads from the SSD, especially in high-dimensional vector databases.

We identify three systemic reasons why existing approaches fail to address this position-seeking overhead on high-dimensional vector databases as follows.

- (1) **Packed layouts force unnecessary vector I/O.** State-of-the-art systems co-locate vectors and neighbor lists on the same SSD page (called packed layout) to exploit search locality [54] and piggyback vector I/O onto edgelist I/O [20, 49]. While efficient for the final ‘convergence’ phase of position seeking, this layout forces the system to load heavy vectors even during the early ‘approach’ phase, when only the edge list should be loaded for graph traversal. More fundamentally, position seeking only needs to find an adequate set of neighbors for connection, not to rank every visited vertex exactly, so much of the loaded vector data never affects which neighbors are chosen. In addition, this packed layout is inefficient for structural updates because it forces the rewriting of the entire neighbor vectors, even when only the edgelists are changed.
- (2) **Static entrance graph stalls navigation.** Many GVS systems utilize an in-memory entrance graph [19, 54], a small sampled (typically 1%) version of the proximity graph that seeds each traversal with well-placed entry points to collapse the graph traversal hops, thereby reducing storage I/O. However, existing update systems either omit the entrance graph entirely or freeze it after construction [20, 57], because concurrent maintenance has its own overhead. A frozen entrance graph is highly likely to drift away from newly inserted regions as insertions accumulate, so traversal entrance points land far from their targets and pull more pages from SSD, diminishing its benefits.
- (3) **Cacheable entrance graph locality goes unexploited.** Caching hot edgelists near entrance graph entries in host memory is a promising direction. However, the packed layout forces to pin vector bytes into every cached page, even though those bytes are only consulted in the final convergence phase, wasting capacity that could hold more edgelists. Worse, a stale entrance graph scatters traversals across the graph, so the cache cannot concentrate on a stable hot set near the entry points.

To this end, we present NAVIS, an on-SSD GVS system leveraging three architectural innovations to reduce position-seeking overhead for high-performance concurrent search and update.

First, NAVIS rethinks the storage layout and the traversal strategy to reduce position-seeking I/O. For the storage layout, we introduce *locality-driven decoupling*, which breaks the packed-page coupling that ties every edge fetch to a full-vector load, while preserving the advantages of packed layouts from existing approaches. To efficiently utilize the layout from locality-driven decoupling, we propose *convergence-aware speculative reranking*, which selectively fetches vectors using convergence-aware early stopping, avoiding vector reads that are unlikely to be promising candidates.

Second, NAVIS supports the in-memory entrance graph under concurrent updates with a lightweight mechanism. Since periodically rebuilding the entrance graph to keep it fresh is too costly, NAVIS instead conducts a lightweight update concurrently

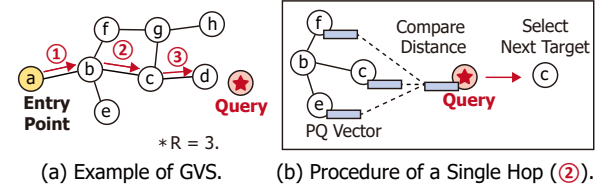


Figure 1: Overview of graph-based vector search (GVS).

by reusing the traversal information that the proximity graph updates already produce. With this, NAVIS maintains high-quality entry points with negligible overhead of less than 1%.

Last, NAVIS exploits the entry-point locality restored by concurrently maintaining entrance graph. By decoupling vectors from edgelists in locality-driven decoupling, NAVIS also increases the effective host-memory cache size. Given the enlarged cache, NAVIS uses a dedicated caching policy that keeps the hot edgelists near entrance graph entries in memory. Together, these accelerate position seeking even with a small cache footprint.

With the above strategies, NAVIS significantly reduces the overhead of position seeking, thereby boosting the performance of concurrent search and update operations on high-dimensional vector databases. We benchmarked NAVIS on multiple large-scale high-dimensional datasets with state-of-the-art GVS update systems, and it improves insertion throughput up to 2.74×. Even under such high insertion concurrency, it reduces search latency by up to 25.26% while increasing throughput by up to 1.37×. Even on the low-dimensional dataset, NAVIS improves insertion throughput up to 2.07× while maintaining the concurrent search performance. We will open-source NAVIS to facilitate adoption in GVS deployments.

2 BACKGROUND

2.1 Graph-Based Vector Search Systems

Graph-based vector search (GVS) is the dominant index design for high-recall vector retrieval over large-scale vector datasets [14, 19, 20, 38, 49, 50]. GVS organizes high-dimensional vectors into a *proximity graph* [38], a sparse structure in which each vertex stores a vector and each edge links two vectors that are close under a distance metric such as L2. The per-vertex out-degree is capped by a hyperparameter R , which controls how exhaustively the graph can be explored at the price of additional I/O and computation. Existing frameworks typically set R between 64 and 96, and recommend filling each vertex up to this out-degree to preserve graph quality.

Fig. 1(a) illustrates the search procedure, which traverses the graph from an *entry point* (vertex a) toward the closest vertex d. A query starts at the entry point and greedily follows the neighbor most similar to the query, converging to the closest vectors in a handful of hops. At each hop, GVS compares the query distances of its neighbors and advances to the closest one (e.g., in Fig. 1(b), vertex b selects c). As computing exact distances against every neighbor with full-precision vectors is costly, GVS utilizes product-quantized (PQ) vectors [28] during traversal, deferring exact distance computation to the vertices it visits.

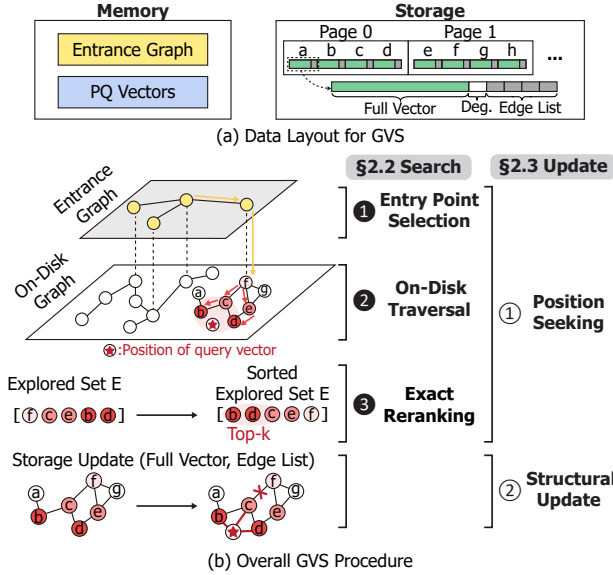


Figure 2: Data layout and overview of on-disk GVS search and update.

2.2 On-Disk Graph-Based Vector Search

In modern vector databases, the proximity graph from Section 2.1 and its accompanying full vectors often far exceed host DRAM. *On-disk GVS systems* [19, 45, 50, 54] therefore offload both to SSD and load them on demand, providing a cost-effective path to high-recall search over millions of high-dimensional vectors [17, 34]. As shown in Fig. 2(a), they keep a small in-memory *entrance graph* and PQ vectors to make traversal cheap, while the full proximity graph and exact vectors are read selectively from the SSD.

Packed storage layout. To minimize the number of storage I/Os, prior frameworks [19, 45, 50, 54] adopt a *packed layout* in which a vector and its edge list reside contiguously within a 4KiB SSD page, formatted as a [Vector] [Degree] [Edge List] pair (see Fig. 2(a)). This packed layout lets the full vector that the final exact top- K selection will need piggyback on an on-disk edgelist read, saving a second I/O. Additionally, in low-dimensional vector databases, multiple pairs can be stored in a single 4KiB page, and this page-level locality could further reduce the number of storage I/Os [54]. However, there are several limitations to this layout that will be revisited in Section 3.

Procedure of search. Using the packed layout, baseline on-disk GVS retrieves the top- K nearest vectors in three stages [19, 50, 54].

① *Entry-point selection.* The system first searches the in-memory entrance graph, a sampled subset of the proximity graph with reduced connectivity (e.g., 1% of vertices, $R=32$) [19, 54]. This process locates well-placed entry points for ②, reducing on-disk hops.

② *On-disk traversal.* Starting from these entry points, the search performs a greedy beam search over the on-disk graph by examining neighbor distances and advancing to the closest one. Distances are computed using PQ vectors rather than full vectors, thereby avoiding heavy computation and I/O overhead. During the traversal, the system manages a fixed-size ($|E_{search}|$) explored candidate pool (Explored Set), and keeps the current top- $|E_{search}|$ closest vertices in that pool until the traversal converges (i.e., no further

candidate produces a closer neighbor). While the system does not use the full vectors during neighbor examinations, it still needs to load the next-visit vertex’s edgelist for traversal under the packed layout, because reading the edgelist also brings the full vector along.

③ *Exact reranking.* Because PQ distance-based examinations are approximate, the system recomputes exact distances for the candidates in E_{search} using their full vectors, which are already loaded into memory via piggybacking. Using these exact distances, the system reranks the E_{search} and returns the final top- K vectors.

To avoid separate full-vector loads for reranking, the system piggybacks vector loads during ②. However, in GVS, there are two traversal phases, ‘approach’ and ‘convergence’ [19]. The traversal rapidly reaches the near-query regions during the approach phase, and it refines the candidate set during the convergence phase to identify the most similar vertices. Since the main reranking target is the candidates in the convergence phase, the piggybacked vector loads from the approach phase are often unnecessary. We will discuss this issue further in Section 3.

2.3 In-Place Update in On-Disk GVS Systems

We discussed the search procedure for on-disk GVS systems, but production deployments also receive continuous updates [10, 13, 17, 22, 29]. Among prevailing update-support strategies, we focus on in-place updates [20, 55], which commit each insertion directly on disk to avoid the search-latency fluctuations of buffered merges [49, 57] and quickly reflect fresh vectors into the index.

Fig. 2(b) illustrates an insertion of vertex $\star$ into a graph of a–g via two steps, ① position seeking and ② structural update. In short, ① finds the neighboring vertices for $\star$, and ② commits the structural change by wiring $\star$ to those neighbors.

① *Position seeking.* Position seeking finds the insertion point such that the neighboring vertices are semantically similar to the new vector, preserving graph quality. Since its core function is finding similar neighbors, this step is identical to the GVS search (①–③). For instance, vertex $\star$ is closest to b, c, and d within the explored set during position seeking (E_{pos}), so those vertices become vertex $\star$ ’s neighbors. However, it requires a much larger explored set ($|E_{pos}| \gg |E_{search}|$) compared to a typical search. This is because a search aims to retrieve the top- K (e.g., 10) vectors, whereas position seeking must accumulate enough candidates to fill the new vertex up to the out-degree R (usually 64–96) to preserve graph connectivity and quality. This leads to much heavier storage I/O than a typical search and interferes with concurrent searches.

② *Structural update.* With the neighbors from ① in hand, the system then wires the new vertex into the graph and commits the changes to storage. The new vertex is connected to neighbors (e.g., $\star$ to b, c, d). This can lead to any existing edge being pruned (e.g., with e, dotted line) if the new connection causes the degree to exceed the maximum out-degree R . The modified pages are then flushed to SSD, completing the in-place insertion.

State-of-the-art in-place updates [20] reduce search interference during structural updates through careful lock isolation, since naively locking the graph structure would otherwise cause a sharp drop in search performance. However, even with this lock isolation, the primary overhead of updates, which is position seeking, remains unaddressed. We further analyze this in Section 3.

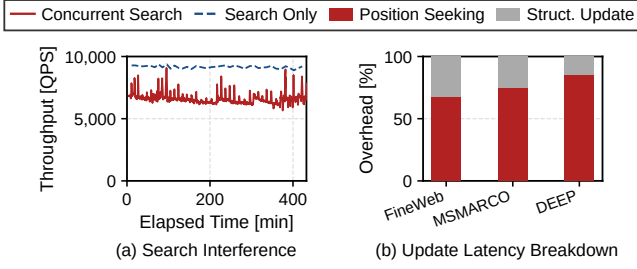


Figure 3: Motivational experiments with OdinANN [20]. (a) Search interference under concurrent updates. (b) Update-latency breakdown showing the position-seeking share.

3 MOTIVATIONAL ANALYSES

Fig. 3(a) shows the search-update interference when updates are conducted on the MSMARCO dataset with the default setup for OdinANN [20] (see Section 9.1 for details). Compared to the search-only throughput, the search throughput under concurrent updates is reduced by 27.89% on average. To find the underlying cause, we profiled the update latency in the same setup. In Fig. 3(b), we decompose the update latency into two stages, Position Seeking and Structural Update. Position seeking accounts for up to 85% of total update time, while structural updates, which were mainly addressed by prior work [20], remain a small fraction.

This confirms that *position seeking* is now the dominant source of search-update interference, and that reducing its overhead is central to efficient concurrent search and update. We identify three main reasons why existing systems fail to address the position-seeking overhead in the following subsections.

3.1 Limitations of Packed Layout

Limitation for position seeking. As mentioned in Section 2, the packed layout is designed to (i) piggyback vector reads on edgelist reads and (ii) exploit page-level locality in low-dimensional settings (e.g., 128-byte vectors). However, the piggybacking under the packed layout incurs unnecessary vector loads. In GVS, traversal is done in two phases, approach and convergence [19]. During the approach phase, the traversal rapidly reaches the near-query region, and during the convergence phase, it locally refines the candidate set of the most similar vertices. Vertices visited in the approach phase are frequently evicted from the explored pool E_{pos} as the traversal converges, and even when they remain in the pool, they are less likely to be included in the final closest vertices. Still, every approach-phase hop must load a full vector because the vector and the edgelist are inseparable on the same page.

Moreover, the nature of position seeking makes this waste even more pronounced. The explored candidate pool is built only to extract adequate neighbors for connection, a set of close vertices plus a handful of long-range shortcuts [50], so exact ranking across the entire pool is unnecessary regardless of how thoroughly it was explored. Fig. 4(a) quantifies this phenomenon in position seeking by measuring the per-insertion read and write volume on MSMARCO across $|E_{pos}| = 64, 100$, and 128 , where $R = 96$. We decompose each per-insertion read and write into four categories, useful vector, wasted vector, edgelist, and padding, where padding is the residual byte volume that the system must read or write to align

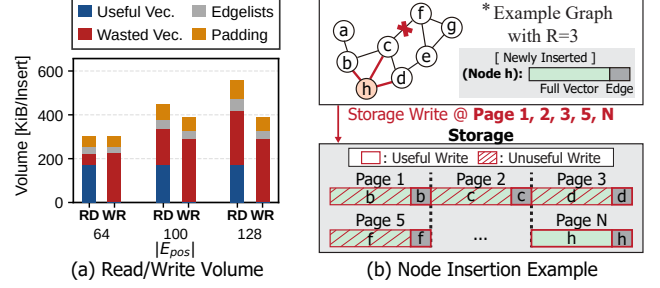


Figure 4: Packed-layout limitations. (a) Wasted vector I/O during per-insertion read and write. (b) A single page accommodates only a single element, and an insertion incurs wasted vector writes across $R+1$ pages in this example.

with the 4 KiB SSD page granularity. Useful and wasted vectors are separated using NAVIS’s PQ-distance-based classifier, which we will describe in Section 5.2 and verify in Section 9.2 to preserve recall on par with existing approaches.

The wasted read volume dominates whenever the search is extensive enough to enter the position-seeking regime ($|E_{pos}| > R$), and its share grows steadily as $|E_{pos}|$ increases. This is due to the nature of position seeking we mentioned earlier, and more extensive seeking sharpens the distinction between the approach and convergence phases, with progressively more vertices visited only as intermediate traversal steps that do not enter the final closest set. Wasted vector reads account for up to 44.34% of the read volume per insertion, and this wasted I/O directly competes with concurrent search bandwidth. Also, the page-level locality benefit of the packed layout breaks down at modern vector dimensions (>2048 bytes), where a single vector and its edgelist already consume an entire page, leaving no room for co-resident records.

Limitation for structural update. Packed layout also creates a write-side problem, as depicted in Fig. 4(b). Inserting a new vertex with out-degree R requires updating R neighbors’ edgelists, and since each neighbor’s page bundles its vector alongside the edgelist, every neighbor update rewrites the full page regardless of what changed. In the figure, this forces a total of $R + 2$ full-page writes (R neighbor updates, 1 inserted-vertex write, 1 pruned-neighbor update), resulting in write traffic far beyond what the structural change actually requires. As the write columns in Fig. 4(a) show, up to 74.23% of the per-insertion write volume is vector data that the structural update never touched.

Unpacking vectors from edgelists into independent stores appears to offer a remedy for the above limitations, as this would let the system selectively load vectors and write only edgelists during updates. A naïve unpacking, however, introduces a competing cost in the form of increased I/O count. This is because the separation removes the benefit of retrieving a vector and its edgelist in a single I/O through piggybacking, now requiring additional vector reads during reranking. Since NVMe SSDs incur a fixed per-request latency on each random 4KiB read, the additional I/O for reranking can erase the bandwidth savings from skipping vector reads. Moreover, without an adequate method to selectively load promising vectors, we cannot exploit the advantage of decoupling. Thus, naïve separation is not a viable remedy, and we need a careful co-design of the decoupled layout and a selective vector-loading algorithm.

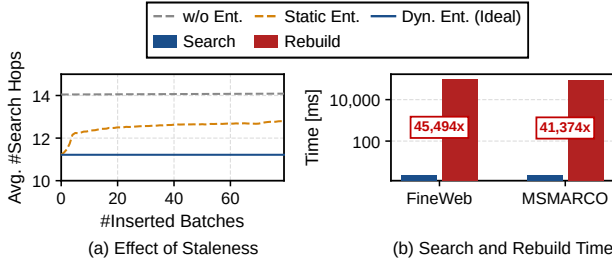


Figure 5: Motivational experiments related to entrance graph. (a) Effect of entrance graph staleness on avg. search hops. (b) Full entrance graph rebuild latency relative to a single search.

3.2 Static Entrance Graph

The entrance graph accelerates traversal by providing well-placed entry points that reduce the number of on-disk hops each search must take. Existing update-supported GVS systems either omit this structure or keep it static throughout the insertion process [20, 49], to avoid the cost of keeping the entrance graph fresh. However, as newly inserted vectors exceed the coverage of the frozen graph, the entry points entrance graph provide drift away from the active regions of the on-disk graph. Fig. 5(a) quantifies this degradation. We insert 0.8M vectors into the 0.5M-vector ImageNet base index and measure the average traversal hops per search query under three configurations, w/o Ent. (no entrance graph), Static Ent. (fixed after build), and Dyn. Ent. (continuously maintained by NAVIS). The static graph initially matches the dynamic one, but its benefit is reduced steadily as insertions push the index beyond the regions it covers, while Dyn. Ent. sustains low hop counts.

To verify why existing update-supported GVS systems omit or freeze the entrance graph, we estimate in Fig. 5(b) the cost of applying periodic rebuilding [50] to the entrance graph to restore entry-point quality. The rebuild takes over 40,000 $\times$ longer than a single search. Naïvely blocking the system during this would cause a severe search throughput drop and a spike in search latency throughout the rebuild window. The entrance graph must therefore be kept up to date by a mechanism that does not stall the system.

3.3 Unexploited Near-Entrance Graph Locality

Caching hot edgelists near entrance graph entries is a promising direction to reduce the edgelist I/O during the approach phase. However, existing frameworks are ill-suited to support this. First, the packed layout wastes cache capacity on co-located vectors. Most page-cache capacity is consumed by vectors that are not utilized during the approach phase, shrinking the effective edgelist cache size¹. Second, a stale entrance graph enlarges the required working set that the approach-phase traversal needs to cover. If the entrance graph is fresh, the approach phase stays short, and the traversed edgelists form a compact hot set. When the entrance graph is stale, the approach phase must cover more intermediate nodes before converging, spreading accesses across a larger set of edgelists. Exploiting this near-entrance graph locality, therefore, requires both a fresh entrance graph that concentrates traversal

¹One could expect that an edgelist cache can be managed at sub-page granularity, but each update must perform a page-wise read-modify-write to keep the storage data consistent, so we manage the cache at page granularity.

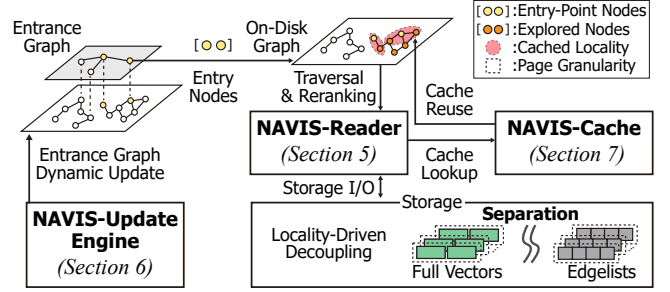


Figure 6: Overview of NAVIS.

near a stable hot set and a layout that frees cache capacity for edgelists, motivating addressing these two problems together.

4 NAVIS OVERVIEW

Fig. 6 shows the overall architecture of NAVIS. Section 3 identified three causes that inflate the position-seeking overhead in existing GVS systems: the packed layout, the static entrance graph, and the unexploited near-entrance graph locality. NAVIS addresses these three causes with three components as detailed below.

- **NAVIS-Reader (Section 5).** To address the packed-layout limitations, NAVIS introduces *locality-driven decoupling*, a fully unpacked on-disk format that separates vectors and edgelists into distinct files while reviving the I/O-saving benefits that the packed layout enjoyed only in low-dimensional datasets. To exploit locality-driven decoupling with adequate vector-read reduction, convergence-aware speculative reranking identifies which vectors are necessary through convergence-aware early stopping, and overlaps speculative I/O submission with exact-distance reranking to hide the per-request latency.
- **NAVIS-Update (Section 6).** To keep the entrance graph fresh without stalling the system, NAVIS incrementally refreshes it during each insertion by reusing the information that position seeking already produces, at no additional traversal cost.
- **NAVIS-Cache (Section 7).** With the layout and the entrance graph addressed, NAVIS introduces an entrance-graph-aware edgelist cache that concentrates host memory on the high-reuse paths near the freshly maintained entry points, while a small admission window filters out edgelists from rare exploration paths before they can pollute the hot region.

5 NAVIS-READER: LAYOUT-SUPPORTED SELECTIVE VECTOR READING

5.1 Locality-Driven Decoupling

To address the unnecessary vector I/O issue demonstrated in Section 3.1, we propose *locality-driven decoupling*, a layout that separates vectors and edgelists while amortizing the additional I/O overhead from the separation by reviving two optimizations originally designed for low-dimensional vectors: page-level locality [54] for reads and out-of-place updates [20] for writes.

Locality-driven decoupling implements this decoupling through two on-disk files and a memory-resident table (Fig. 7(a)). Specifically, the *edgelist file* packs multiple edgelists per page, the *vector file*

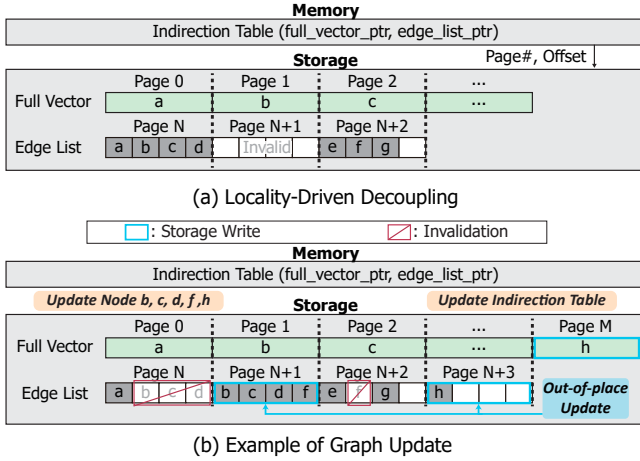


Figure 7: Overview of locality-driven decoupling with the same graph as other figures and an example update process.

stores full-precision vectors, and the *host-memory indirection table* maps vertex IDs to their physical SSD locations.

The edgelist file in locality-driven decoupling enables page-level locality without being affected by the vector dimension because the decoupling lets edgelists gather densely without co-located vectors. In Fig. 7(a), multiple edgelists share a single page regardless of vector size. The nearby vertices’ (a, b, c, and d) edgelists reside on one page, so reading a’s edgelist during traversal fetches b, c, and d in the same I/O, reducing the total traversal I/O count.

Locality-driven decoupling also enables updates with a much smaller write volume and fewer page writes. As the vectors are decoupled from the edgelists, an update writes only the target vector to the vector file and skips the neighbors’ vectors entirely. In Fig. 7(b), only h’s own vector is written to the vector file, since unchanged neighbors’ vectors are never touched. Also, locality-driven decoupling writes only the modified edgelists out of place, as illustrated in Fig. 7(b). For instance, when h is inserted, the edgelists of b, c, d, and f are gathered and written into a single new page. Their old edgelist entries are invalidated, and the indirection table is updated to point to the new locations². This reduces five full-page writes to two edgelist-page writes and one vector-file write, decoupling vector write volume from the graph’s out-degree R and thereby improving SSD endurance.

One concern is that page-level locality could be diminished during insertion. Fortunately, the out-of-place write also preserves page-level locality through insertions. Co-updated vertices tend to be adjacent in the graph and are likely to be co-traversed in future queries, so gathering them on a single new page maintains locality without explicit reorganization. For the initial page-locality-aware placement, we utilize the same greedy placement as [54]. The indirection table itself adds little overhead, since it stores only a (page-number, offset) pair per vertex for both edge and vector locations, which is negligible relative to the in-memory PQ vectors (Section 9.6).

²The out-of-place update prefers writing to fully invalidated pages when possible, thereby reducing overprovisioned storage usage. In Fig. 7(b), the new edgelist page for b, c, d, f reuses a fully invalidated page.

Algorithm 1 Convergence-Aware Speculative Reranking (CASR).

Input: Query q , candidate set sorted by PQ distance E_{pos}^{sorted} , top- K size K , vector loading group size s (calibrated at warm-up).

Output: Dict with (vertex_id, exact distance) pairs (D), Top- K neighbors of q (T).

```

1:  $D, T \leftarrow \emptyset, \emptyset$  ▷ Init exact distance set and top- $K$  set
2:  $idx \leftarrow 0$  ▷ Offset in  $E_{pos}^{sorted}$  for vector loading
3:  $VecLoad(E_{pos}^{sorted}, idx, s); idx += s$  ▷ Load pipeline start
4: while PrevVecLoad is non-empty do
5:   WaitPrevVecLoad()
6:   if  $idx < |E_{pos}^{sorted}|$  then ▷ Speculative next I/O
7:      $s \leftarrow \min(s, |E_{pos}^{sorted}| - idx)$ 
8:      $VecLoad(E_{pos}^{sorted}, idx, s); idx += s$ 
9:   end if
10:  for all (vertex_id, vector)  $\in$  PrevVecLoad do
11:     $d \leftarrow L2Distance(q, vector)$ 
12:     $D \leftarrow D \cup \{(vertex\_id, d)\}$  ▷ Exact distance compute/store
13:  end for
14:   $T^{new} \leftarrow TopK(D, K)$  ▷ Get new top- $K$  set from the updated  $D$ 
15:  if  $Equal(T, T^{new})$  then break ▷ Convergence test
16:  else  $T \leftarrow T^{new}$  endif
17: end while
18: return ( $D, T$ )

```

5.2 Convergence-Aware Speculative Reranking

While our layout enables decoupling, realizing vector I/O savings necessitates *convergence-aware speculative reranking*, a mechanism leveraging two key insights to selectively load vectors.

First, the underlying goal of position seeking differs from that of a typical search. Search must rerank a small explored set ($|E_{search}|$, typically 10–80) tightly to surface the exact top- K , so every candidate’s exact distance matters. Position seeking, in contrast, uses a much larger explored pool ($|E_{pos}| \gg |E_{search}|$) only to extract the top- R neighbors that will be wired into the new vertex. The large pool is necessary for thorough exploration, but its role is only to surface adequate neighbors for connection. This follows Vamana’s [50] philosophy of intentionally combining close neighbors with long-range shortcuts for efficient traversal. Exhaustive reranking is therefore unnecessary, as the goal is identifying candidates rather than fully ranking all visited vertices.

Second, PQ distances computed during traversal provide a strong signal for identifying close-neighbor candidates. Candidates with the smallest PQ distances are most likely to be the closest under exact distances as well, so PQ ranking gives a reliable order for issuing full-vector loads to identify the close neighbors.

Leveraging these insights, convergence-aware speculative reranking runs as the reranking step at the end of position seeking, replacing the conventional reranking that loads full vectors for every candidate in E_{pos} . Under typical settings ($|E_{pos}| = 100$, $R = 96$), nearly the entire pool becomes wired as the new vertex’s edges, yet only the close-neighbor portion requires exact ranking to be reliably identified. To reduce the vector load by only reranking the promising close neighbors, we load full vectors based on PQ-distance order and track the top- K closest candidates among them by exact distance, where K is the search’s top- K size (e.g., $K = 10$). Once the top- K stops changing, the close-neighbor portion is settled, and the new vertex’s remaining R edges fall to shortcut slots. This strategy

adaptively adjusts the number of exact-distance computations per query. When the query is near many candidates, PQ distances fail to differentiate them, and convergence-aware speculative reranking fetches more vectors until the top- K set stabilizes.

Algorithm 1 details the procedure, which implements the above process with low I/O submission overhead, since issuing one I/O at a time per neighbor would be too costly. First, it loads vectors in groups of size s and pipelines each group's I/O submission with the exact-distance computation of the previous group, hiding submission latency through speculative loads (lines 6–9). Second, after each group is loaded and exact distances are computed, it updates the top- K and compares it against the previous one (lines 10–16). If they are identical, the close-neighbor portion has settled, and convergence-aware speculative reranking terminates, returning both the exact-distance set D and the converged top- K .

For position seeking, D provides exact distances for the close-neighbor portion, and the new vertex's remaining R edges are filled with other neighbors based on PQ distances (Section 5.3), since shortcut slots only require diversity rather than exact closeness. Convergence-aware speculative reranking also runs for search by substituting E_{pos}^{sorted} with E_{search}^{sorted} in Algorithm 1, with the top- K set T used directly as the result. The vector I/O reduction gain is smaller than for position seeking since the smaller $|E_{search}|$ leaves less room for early stopping, consistent with the sub-saturation regime ($|E_{pos}| = 64$) in Fig. 4(a). We use different s for the search and position seeking paths because $|E_{pos}| \gg |E_{search}|$.

Group size s is calibrated once at warm-up using 100 queries. To obtain s , we initially set $s=1$ to get the distribution of the number of vectors needed for top- k to stabilize across all queries. We then set s to the P25 of the distribution, balancing convergence-detection granularity against I/O submission overhead. We find P25 to be a good sweet spot, as extreme values are clearly suboptimal since $s=|E_{pos}|$ degenerates to a full fetch of $|E_{pos}|$ vectors and very small s submits I/O too frequently, as shown in Section 9.8.

5.3 Update Example

Fig. 8 illustrates the detailed update procedure with NAVIS-reader, covering position seeking and structural update on the same example graph used in earlier figures. We omit the entry-point selection step for brevity. ① **On-disk traversal**. To insert a new query vector, we traverse the on-disk graph and add the visited vertices to the explored set E_{pos} ($|E_{pos}| = 20$ in this example, while the actual setting is much larger). During this step, we read only edgelist pages from storage and rely on host-memory PQ vectors for distance comparisons, exploiting locality-driven decoupling's separated edgelist pages to avoid full-vector I/O. ② **Exact reranking with convergence-aware speculative reranking**. After traversal, we sort E_{pos} by PQ distance to obtain $E_{pos}^{sorted} = \{b, e, c, d, a, f, g, \dots\}$. Using a group size of $s = 2$ and $K = 3$ in this example, we load full vectors group by group, overlapping each group's I/O with the previous group's exact-distance computation, and maintain the running top- K as exact distances accumulate. We stop once the top- K stops changing. In this example, the top- K set $\{b, c, d\}$ stabilizes after Stage 2, so convergence-aware speculative reranking terminates with 6 full-vector loads instead of 20, significantly reducing vector I/O. The remaining $R - 6$ neighbors needed

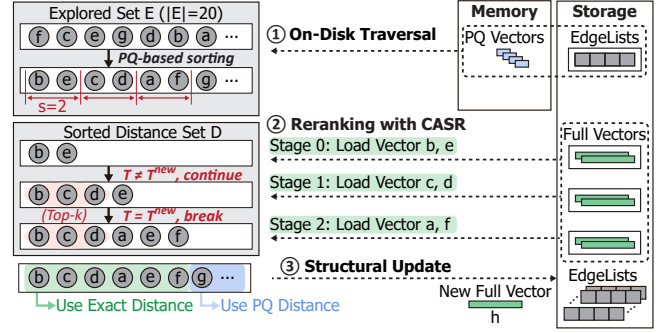


Figure 8: Detailed example of NAVIS-reader.

Algorithm 2 NAVIS-Update

Require: Query vector q , entrance graph G_{ent} , entrance graph explored set E_{ent} , on-disk explored set during position seeking E_{pos} , subgraph ratio r_{ent}

```

1: if  $|G_{ent}|/|G| < r_{ent}$  then ▷  $G_{ent}$  needs more coverage
2:    $E_{inter} \leftarrow E_{pos} \cap G_{ent}$  ▷ Select nbr candidates only from  $G_{ent}$ 
3:    $N_{ent} \leftarrow \begin{cases} E_{inter} \cup (E_{ent})_{[1:R-|E_{inter}|]}, & |E_{inter}| < R, \\ (E_{inter})_{[1:R]}, & |E_{inter}| \geq R \end{cases}$ 
4:    $q.nbr \leftarrow N_{ent}$ 
5:   for  $p \in q.nbr$  :  $p.nbr \leftarrow \text{prune}(p.nbr \cup q)$ 
6:   Lock( $G_{ent}$ )
7:    $G_{ent} \leftarrow G_{ent} \cup q$ 
8:   for  $p \in q.nbr$  :  $p.nbr \leftarrow p.nbr$ 
9:   Unlock( $G_{ent}$ )
10: end if

```

for the new vertex's edges fall to shortcut slots and are filled using deflated PQ vectors. ③ **Structural update**. With the neighbor list assembled, we commit the structural change by writing only the new vertex's full vector and the modified edgelist pages, benefiting from locality-driven decoupling's decoupled writes.

6 NAVIS-UPDATE: DYNAMICALLY UPDATING THE ENTRANCE GRAPH

As shown in Section 3.2, even a moderate number of insertions can make a static entrance graph stale enough to lose its benefits, and rebuilding the entrance graph incurs severe overhead. To address this issue, we propose dynamically updating the entrance graph incrementally with low overhead. The challenge is that inserting the new vector into the entrance graph itself requires a dedicated position-seeking step to build its neighbor list.

NAVIS-update addresses this by reusing the work already performed during on-disk insertion. We invoke NAVIS-update immediately after the on-disk insertion completes, where the entrance graph explored set E_{ent} from entry-point selection and the on-disk explored set E_{pos} are available for reuse. This avoids the need for an additional position-seeking step on the entrance graph.

Algorithm 2 depicts the NAVIS-update algorithm. The update is triggered when the size of entrance graph ($|G_{ent}|$) falls below a threshold ratio of the on-disk graph size ($r_{ent} \times |G|$). Following prior work [19, 54], we set $r_{ent} = 0.01$ (line 1). When an update is triggered, we first find the intersection of E_{pos} and G_{ent} to obtain E_{inter} , which retains only vertices already in the entrance graph

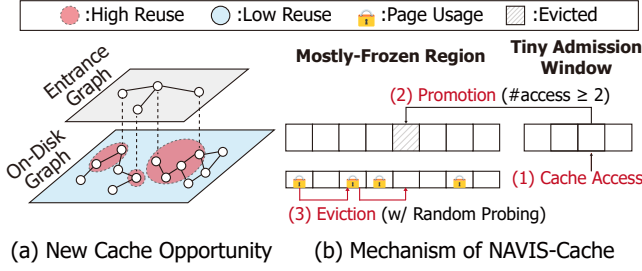


Figure 9: Locality induced by the dynamic entrance graph and the overview of NAVIS-cache.

(line 2). This ensures that no vertex other than the new vector q is added to G_{ent} . We use E_{inter} and E_{ent} to build a neighbor list for the new vector q , capped at R_{ent} entries (lines 3-4). We prioritize E_{inter} among them because its vertices are typically closer to the query. Afterward, reciprocal links to q are added from its neighbors, whose list can be optionally pruned to meet the maximum degree limit R_{ent} (line 5).

Despite performing updates on the in-memory entrance graph, NAVIS-update incurs minimal lock overhead during insertion (lines 6-9). In practice, adding a single vertex with a handful of neighbors into G_{ent} takes only microseconds, and only a small fraction of queries are promoted into the entrance graph. Consequently, lock contention on G_{ent} is negligible even with tens of concurrent insertion threads, and NAVIS-update introduces less than 1% throughput overhead, as shown in Section 9.3.

7 NAVIS-CACHE: PRIORITIZED IN-MEMORY CACHING OF THE PROXIMITY GRAPH

Section 3.3 established that exploiting near-entrance graph locality requires both a fresh entrance graph to concentrate traversals and a decoupled layout to reduce unnecessary cache usage from vectors. NAVIS-Update and locality-driven decoupling together satisfy both conditions, and with a fresh entrance graph, queries repeatedly visit the vertices near entry points, creating a concentrated hot set as shown in Fig. 9(a).

To exploit this edgelist locality, a straightforward host-memory LRU cache managing edgelist pages from locality-driven decoupling might seem promising. However, LRU performs poorly in this setting for two reasons. First, large portions of the on-disk graph are explored with low reuse during traversal, and these edgelists rapidly evict truly hot entries near entry points. Such pollution sharply degrades LRU’s hit rate under realistic workloads (Section 9.7). Second, convergence-aware speculative reranking allows the host memory edgelist cache (e.g., 16GB) to hold tens of millions of edgelists, making software-based LRU maintenance expensive.

To exploit entry point locality while avoiding LRU’s pitfalls, we design NAVIS-cache. Fig. 9(b) illustrates its structure. Inspired by frozen-cache designs [12, 46], NAVIS-cache partitions into a *mostly-frozen region* (90%) that preserves the hot region near entry points and protects it from eviction, and a *tiny admission window* (10%) that provides lightweight, usage-aware admission control.

An edgelist must be accessed at least twice within the admission window to be promoted into the mostly-frozen region. The window itself is managed by LRU because of its substantially lower

overhead compared to frequency-based policies such as LRU. This rule effectively keeps hot edgelists near entry points while filtering out one-off accesses from long exploration paths, outperforming a full LRU cache as shown in the evaluation (Section 9.7). As NAVIS-update refreshes the entrance graph during insertions, the admission window naturally incorporates newly relevant edgelists.

For the mostly-frozen region, NAVIS-cache uses randomized eviction, selecting a random candidate and, if that entry is currently in use, performing a small number of random probes (up to eight by default) to find an alternative. This avoids expensive tracking structures while preventing evictions of actively used entries.

8 IMPLEMENTATION

NAVIS follows OdinANN’s [20] in-place update concurrency model for the search/update lock control, with the I/O-path and cache-coordination modifications described as follows.

8.1 I/O Path Optimization

`io_uring` [1] delivers high-throughput asynchronous SSD I/O by batching submissions and completions on per-thread rings, eliminating the per-request system-call overhead of synchronous read/write paths. NAVIS therefore issues all storage I/O through `io_uring`. However, naïve use still incurs per-call kernel overhead under the large I/O bursts of position seeking, so NAVIS carefully utilizes `io_uring` to keep this overhead near-constant. Specifically, each worker thread owns a private ring of depth 256 and pre-registers both the edge and the vector files via `io_uring_register_files`, so every submission carries an `IOSQE_FIXED_FILE` index instead of a raw file descriptor. Each group of s vector reads from the candidate pool is committed in a single `io_uring_submit_and_wait` call, and completions are harvested in bulk via `io_uring_peek_batch_cqe`, paying a single enter-syscall per group instead of one per submission/completion.

8.2 NAVIS-Cache Control Details

Concurrent search and update force NAVIS-cache to coexist with two write-side mechanisms, a read-modify-write (RMW) page cache that holds dirty pages until they are flushed, and locality-driven decoupling’s out-of-place edge updates that relocate edgelists across pages. NAVIS addresses both with the design described below.

The interaction between NAVIS-cache and the RMW page cache is governed by a single rule. NAVIS-cache is a write-through cache for clean read-side traffic, and the RMW page cache is the only structure that ever holds a dirty page, so eviction from NAVIS-cache never has to commit data, and the RMW path never has to negotiate freshness with NAVIS-cache. On a search or position-seeking miss, the worker probes the RMW cache first to pick up any in-flight modifications, falls through to NAVIS-cache for a clean hot page, and only issues storage I/O on a double miss, after which the page is admitted to the RMW cache and then to NAVIS-cache’s admission window. Structural update operates exclusively on the RMW cache, since it pins all dirty pages with reference counts and releases the page lock only after the RMW page has been written to storage and any matching entry in NAVIS-cache has been refreshed in place. This ordering prevents NAVIS-cache from ever holding a page newer than the disk.

The out-of-place edge updates in locality-driven decoupling add a second consistency obligation, since a vertex’s edgelist may move to a freshly allocated edge page, rendering the old slot stale. We enforce freshness through the indirection table rather than through cache metadata. A reader resolves $\text{id} \rightarrow \text{edge_list_ptr}$ from the indirection table before consulting any cache, so a hit on a relocated id targets the new page and the stale page is never queried again. When all slots in a 4 KiB edge page have been invalidated, the indirection layer issues an eviction hint to NAVIS-cache for that page number, so the page is reclaimed before the cold-path randomized eviction would touch it. Per-vertex tombstones and version counters are unnecessary, since invalidation is page-grained and the indirection table already provides the freshness guarantee. This keeps NAVIS-cache lock-free on the read path and free of dirty-page state management.

9 EVALUATION

9.1 Experimental Setup

Hardware. We used the following single server for all evaluations:

- **CPU:** AMD Ryzen 9 7950X3D (16C 32T) @ 4.2–5.7GHz
- **RAM:** 128GB DDR5 (4× DDR5 5600MT/s)
- **SSD:** 1× Micron Crucial PCIe 5.0 T705 4TB
- **OS:** Ubuntu 24.04 LTS

Datasets. We evaluated NAVIS on four datasets in total and conducted the main experiments on three datasets spanning different dimensionalities and database scales, as shown in Table 1. FineWeb document vectors are embedded with EmbeddingGemma [53], while ImageNet images are embedded with CLIP [47]. DEEP uses low-dimensional vectors generated by compression [2], which we include to demonstrate that NAVIS performs well even on low-dimensional datasets. ImageNet is used only for testing a production baseline (QDRANT [45]) and a clustering-based baseline (SPFRESH [56]), since those baselines incur significantly lower recall or throughput than the main GVS baselines. We inserted vectors equivalent to 20%, 100%, 20%, and 30% of the base index size for FineWeb, MSMARCO, DEEP, and ImageNet, respectively. All datasets use $R = 96$ for graph construction, following prior work [20, 50]. Image-based embedding vectors (i.e., DEEP and ImageNet) require much larger $|E_{\text{pos}}|$ values of 200 and 128, respectively, due to their high similarity among vectors. We randomly selected 10,000 queries from each dataset following standard methodology [20, 50]. For search, we targeted high recall levels of .95, .95, .85, .95 for FineWeb, MSMARCO, DEEP, and ImageNet, respectively, using $|E_{\text{search}}| = 40$ for all datasets. We measured recall with top-10 selection (i.e., Recall@10). DEEP shows lower target recall at the same candidate pool size because it has fewer dimensions than other datasets.

Baselines. We mainly compared NAVIS against three GVS-based baselines, two state-of-the-art GVS frameworks supporting dynamic insertions, and another variant that we enabled the NAVIS-cache as a stronger baseline:

- **FRESHDISKANN** [49]: a buffered-insertion design using a host-side buffer that flushes and merges into the on-disk graph once the buffer reaches 6% of the base graph size.
- **ODINANN** [20]: a concurrent insertion baseline that performs in-place updates preventing search fluctuation.

Table 1: Dataset Configurations Used in NAVIS Experiments.

Dataset	Dim. (float)	#Vectors	Type	$ E_{\text{search}} $	$ E_{\text{pos}} $	PQ Bytes
FineWeb [37]	768	60,000,000	Text	40	100	128
MSMARCO [3]	768	100,000,000	Text	40	100	128
DEEP [2]	96	120,000,000	Image	40	200	32
ImageNet [32]	512	1,300,000	Image	40	128	64

- **ODINANN+\$**: ODINANN with NAVIS-cache. Since ODINANN retains the packed layout, NAVIS-cache must cache packed pages (vector co-resident with edgelist), so its effective edgelist cache size is smaller than NAVIS’s for the same memory budget.
- **NAVIS (Ours)**: a high-performance concurrent insertion framework that reduces unnecessary vector reads and employs dynamic entrance graph with an efficient cache.

All systems used a beamwidth of four. Since FRESHDISKANN employs buffered insertion and merges the buffered graph every 6% increase, we derived its insertion throughput by averaging it over time. We tested only FRESHDISKANN as the representative baseline for buffered insert-based approaches [49, 57], since these approaches are well known for concurrent search fluctuations under large updates [20], as our evaluations also confirm. Prior work [19, 54] that does not support concurrent updates is excluded.

We also benchmarked other baselines, including production baselines (i.e., QDRANT [45]) and a cluster-based baseline (i.e., SPFRESH [56]) to show that they often fail to provide high-throughput search and insertion with high recall. For this reason, we only tested GVS systems in the main evaluations.

Other settings. We configured 22 search and 10 insertion threads by default, following prior work [20]. For overall performance evaluations, we report results for up to 12 hours of insertions. For other experiments, we present measurements from the first 100K insertions, and when testing search-only tasks, cache-based baselines are warmed up by running the full 10,000 query search ten times. We used a 16GB host memory cache for NAVIS-cache in both ODINANN+\$ and NAVIS to ensure a fair comparison, matching the memory usage of FRESHDISKANN, which uses much more host memory for its insertion buffer. We enabled the entrance graph for all GVS baselines, with ten entry points each.

9.2 Concurrent Search-Insert Performance

In Fig. 10–Fig. 12, NAVIS delivers substantial improvements over the baselines, showing up to 2.74× higher insertion throughput. Even under such high concurrent insertion, NAVIS achieves better search performance, up to 1.37× higher average query per second (QPS), and up to 25.26% lower average latency, while consistently maintaining stable recall. We discuss the general trends in the FineWeb results, then highlight the key observations for each remaining dataset. For all datasets, NAVIS’s curves end earlier than the baselines’ because its high insertion throughput finishes the update workload much faster.

FineWeb. On the FineWeb dataset (Fig. 10), by reducing position-seeking overhead, NAVIS delivers 1.36/1.60/1.54× higher insertion throughput compared to FRESHDISKANN, ODINANN, and ODINANN+\$, respectively. With reduced interference on search, NAVIS

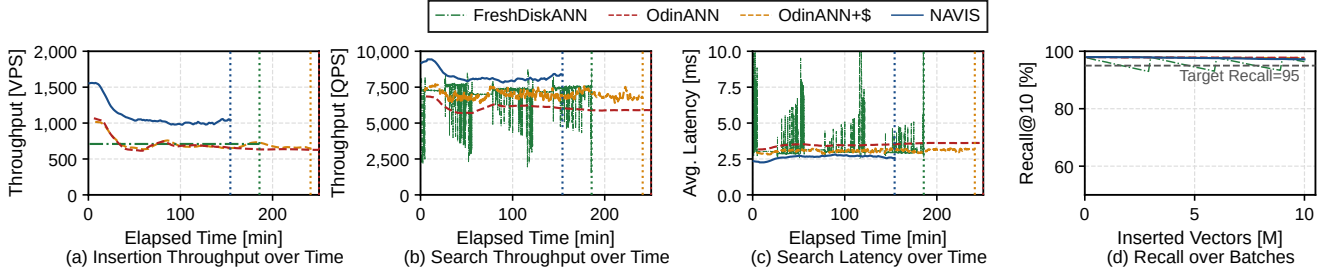


Figure 10: Update throughput and search performance under concurrent updates of baselines and NAVIS on the FineWeb dataset.

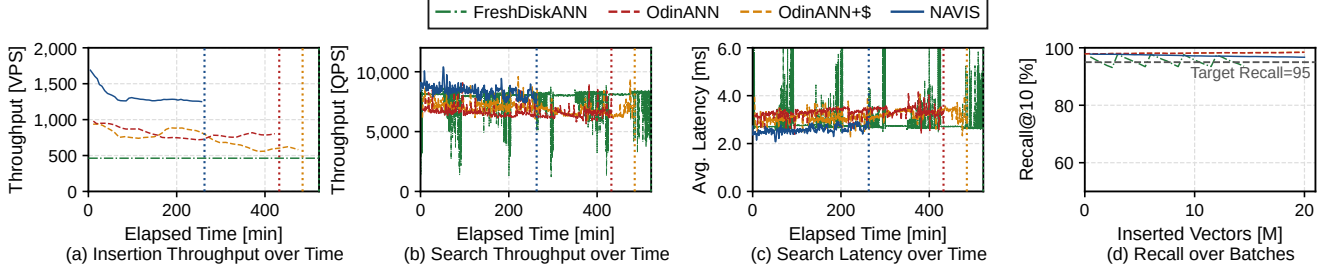


Figure 11: Update throughput and search performance under concurrent updates of baselines and NAVIS on the MSMARCO dataset.

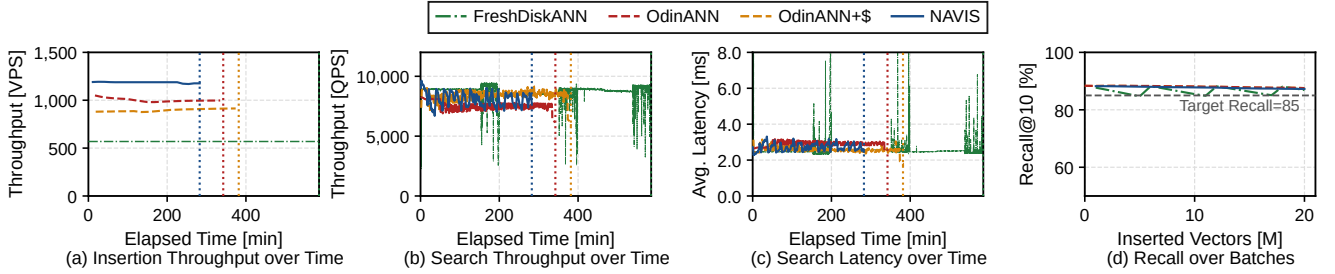


Figure 12: Update throughput and search performance under concurrent updates of baselines and NAVIS on the DEEP dataset.

achieves 1.17/1.37/1.19 $\times$ higher average QPS and 18.98/25.26/15.06% lower average latency compared to FRESHDISKANN, ODINANN, and ODINANN+\$, respectively. The NAVIS-cache helped ODINANN+\$ to some degree, but without the locality-driven decoupling, the performance improvement over ODINANN is limited, especially for insertion throughput, due to redundant vector I/O and the small effective cache size from the packed layout. FRESHDISKANN exhibits the buffered-merge instability previously reported by [20], where its worst-case search QPS drops 79.1% relative to its average during merge windows, visible as the spikes in Fig. 10(b).

Regarding recall, FRESHDISKANN drops before each buffered batch is merged into the on-disk graph and recovers after the merge. NAVIS, ODINANN, and ODINANN+\$ converge to similar recall (near 97%), with FRESHDISKANN reaching the same level after each merge, all comfortably above the 95% target.

MSMARCO. On the MSMARCO dataset, NAVIS provides 2.74/1.63/1.83 $\times$ insertion throughput increase compared to FRESHDISKANN, ODINANN, and ODINANN+\$, respectively. In terms of search statistics, NAVIS provides 1.06/1.26/1.20 $\times$ QPS increase and 9.43/21.76/17.78% latency reduction compared to FRESHDISKANN, ODINANN, and ODINANN+\$, respectively. The search QPS gain over FRESHDISKANN is smaller than for FineWeb because NAVIS’s insertion throughput on MSMARCO far exceeds

FRESHDISKANN’s, interfering more with concurrent search. Even so, NAVIS provides more stable and better search performance than FRESHDISKANN. ODINANN+\$ benefits from NAVIS-cache for search, but due to the low hit rate from the small effective size with the packed layout, it shows lower insertion throughput than ODINANN. Regarding recall, FRESHDISKANN again shows fluctuation (93%–97%) as in the FineWeb case, and NAVIS (97%) matches FRESHDISKANN’s max recall while sitting 1% below ODINANN (98%), still comfortably above the 95% target.

DEEP (low-dimensional dataset test). While NAVIS mainly targets the inefficient vector I/O and effective cache size issues on high-dimensional vector databases, we also benchmarked NAVIS on a low-dimensional dataset. Even with this dataset, NAVIS achieves 2.07/1.21/1.35 $\times$ insertion throughput increase compared to FRESHDISKANN, ODINANN, and ODINANN+\$, respectively. Even under this high insertion throughput and despite the baselines being optimized for low-dimensional datasets, NAVIS’s search performance is similar or slightly lower than the baselines, providing 0.93/1.08/0.96 $\times$ QPS over FRESHDISKANN, ODINANN, and ODINANN+\$, respectively. This advantage stems from NAVIS’s reduced vector I/O and an optimized caching strategy that exploits the entrance graph locality. ODINANN+\$ confirms this, with higher search but lower insertion throughput than ODINANN since it does not

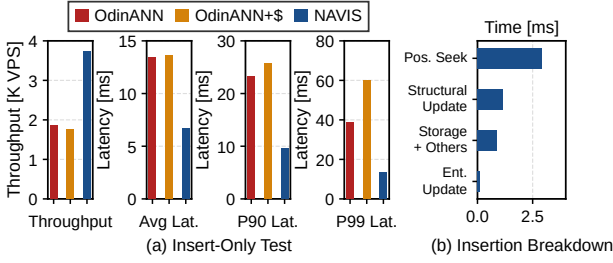


Figure 13: Insert-only test and insert time breakdown.

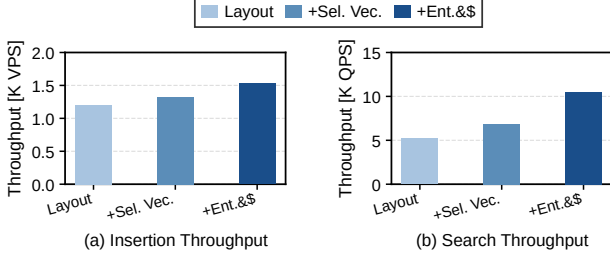


Figure 14: Effect of NAVIS components.

reduce vector I/O. Regarding recall, NAVIS achieves recall identical to ODINANN, while FRESHDISKANN shows the same fluctuating pattern as in other datasets.

9.3 Insert-Only Test and Time Breakdown

Since NAVIS aims to reduce update overhead during position seeking, we also conducted an insert-only benchmark (32 insert threads and zero search threads) on MSMARCO (Fig. 13(a)) and broke down the insert time during the run in Fig. 11 (Fig. 13(b)). With its significant vector I/O reduction and effective cache, NAVIS achieves 2.01/2.12 $\times$ higher throughput than ODINANN and ODINANN+\$, respectively, while substantially reducing insert latency, especially on tail latencies (P90/P99, up to 65.74% over ODINANN). From the breakdown, the position-seeking overhead becomes relatively smaller than in Fig. 3 (74.83% $\rightarrow$ 58.24%) under NAVIS’s vector I/O reduction strategy. Also, Ent. Update, which denotes the entrance graph update overhead of NAVIS-update, accounts for less than 1% of the total time, confirming that NAVIS-update overhead is negligible.

9.4 Ablation Study

To evaluate the contribution of each component of NAVIS, we performed an ablation study on the MSMARCO dataset when running concurrent search-insert, as shown in Fig. 14. Starting from the baseline locality-driven decoupling (Layout), we incrementally enabled additional components. **Layout**: Because locality-driven decoupling alone cannot reduce vector read I/O volume, it yields the lowest search and insertion throughput among the configurations. **+Sel. Vec.**: Adding NAVIS-reader increases insertion and search throughput by 1.10 $\times$ /1.32 $\times$ over Layout, confirming that reducing unnecessary vector reads is a key driver of NAVIS’s performance gains. **+Ent.&\$**: Adding NAVIS-update and NAVIS-cache on top of +Sel. Vec increases insert/search throughput by 1.16 $\times$ /1.53 $\times$, since locality-driven decoupling enables a larger effective cache size and NAVIS-cache exploits the locality near the freshly maintained entrance graph entries.

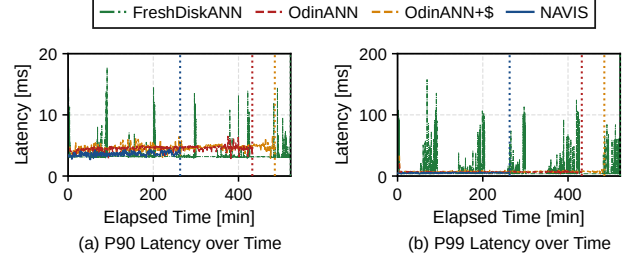


Figure 15: Search tail latency (P90/P99) profiling results.

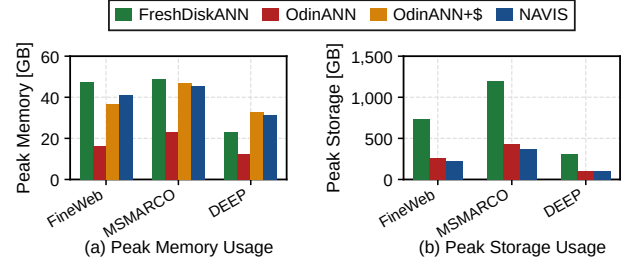


Figure 16: Memory and storage consumption.

9.5 Search Tail Latency Analysis

Fig. 15 shows the P90/P99 latency profile on MSMARCO from the experiments in Section 9.2. NAVIS reduces the maximum P90 latency by 64.01%/65.78% relative to FRESHDISKANN and ODINANN, and the maximum P99 latency by 92.22%/84.78%, respectively. This improvement primarily stems from NAVIS’s reduction in vector I/O, which dominates long-latency outliers under concurrent updates.

9.6 Memory and Storage Usage

Fig. 16 shows the peak memory consumption and storage usage of NAVIS compared to baselines. FRESHDISKANN consumes relatively more host memory than the others on high-dimensional datasets because it uses a host-memory buffer for merging. We configured ODINANN+\$ and NAVIS with a 16GB NAVIS-cache so that their peak host memory matches FRESHDISKANN on these datasets. ODINANN shows the lowest consumption since it has no cache. On DEEP, FRESHDISKANN’s host memory drops below ODINANN+\$ and NAVIS because the low-dimensional vectors yield a smaller insertion buffer, while ODINANN+\$ and NAVIS retain the 16GB NAVIS-cache for consistency across datasets. Regarding storage, FRESHDISKANN utilizes significantly more storage because it adopts a double-buffered storage scheme during merge. NAVIS and ODINANN show much lower storage usage because they apply in-place storage updates, with NAVIS slightly below ODINANN because it eliminates per-page padding.

9.7 Search-Only and Cache Policy Test

While NAVIS primarily targets update overhead during position seeking, NAVIS-reader and NAVIS-cache also benefit search-only workloads, as shown in Fig. 17(a) on the MSMARCO dataset with 32 search threads and zero insert threads. Across all recall levels with various E_{search} , NAVIS provides higher QPS than ODINANN and ODINANN+\$, up to 1.60 $\times$. We additionally tested NAVIS-cache’s policy choices, as shown in Fig. 17(b). With locality-driven decoupling,

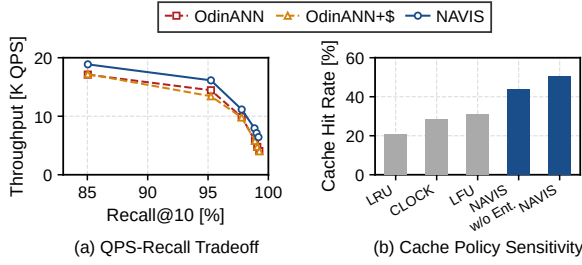


Figure 17: Search-only test and cache policy comparison.

a 16GB cache size yields over 90% cache hit across all baselines, so we conducted this experiment with the cache size forced to 4GB. We measured cache hit rates on the FineWeb dataset under several policies, including well-known LRU, FIFO with second chance (CLOCK), and LFU. We also evaluated NAVIS-cache without NAVIS-update (NAVIS-wo Ent) to assess the impact of NAVIS-update on improving locality near entry points. As expected, conventional policies suffer from cache pollution caused by low-reuse edgelists and fail to retain near-entry-point structures in the cache. In contrast, NAVIS-cache effectively prevents such pollution and improves cache hit rate by 19.28 pp over LFU. Moreover, NAVIS-update further enhances cache locality, providing an additional 6.78 pp cache hit improvement over the configuration without NAVIS-update.

9.8 Group Size Sensitivity and Comparison with Other Baselines

In Fig. 18(a), we tested the sensitivity of NAVIS to group size s with the insert-only test using 32 threads. As expected, $s = 1$ submits I/O too frequently and $s = |E_{pos}|$ loads all vectors in the candidate set, both showing lower insertion throughput than other s choices. However, both extremes remain much faster than the baselines in Section 9.3, since $s = 1$ already cuts the vector load substantially and $s = |E_{pos}|$ still avoids per-hop full-vector loads during traversal, loading vectors only for the reranking pass. We also compared NAVIS against a production baseline (QDRANT [45]) and a clustering-based baseline (SPFRESH [56]) on the ImageNet dataset in Fig. 18(b). For QDRANT, we used the new HNSW [38] features with storage from version 1.16.3, which use a strategy similar to NAVIS and the GVS baselines. QDRANT shows high recall, but its search and insert throughput are both significantly lower. SPFRESH achieves high insert throughput from its batched, cluster-based design, but falls short on recall and search throughput. Since these baselines fall short on at least one of recall, search, or insert performance, the GVS baselines remain our focus in the main evaluation.

10 RELATED WORK

While numerous graph-based vector search solutions have been explored across various platforms [7, 9, 18, 27, 30, 31, 38, 41, 52, 54, 58–60, 62, 64], on-disk solutions [16, 19, 26, 30, 44, 48, 50] offer a low-cost, scalable alternative for large-scale vector databases by storing the full graph on storage devices while keeping compressed vectors in memory for efficient traversal. Several recent systems [20, 36, 49, 55, 61] have addressed dynamic scenarios with runtime graph updates. FreshDiskANN [49] supports inserts and deletes through buffered updates, which incurs significant disk

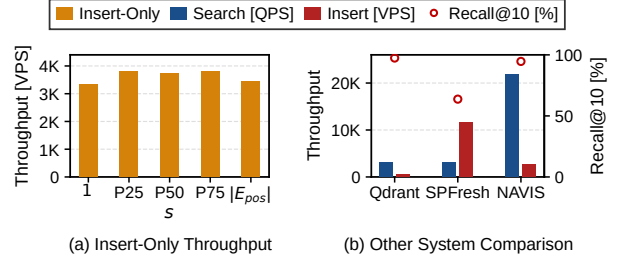


Figure 18: Sensitivity of group size and comparison with production (QDRANT) and cluster-based (SPFRESH) baselines.

write overhead. OdinANN [20] mitigates this through write over-provisioning with approximate concurrency control. CleANN [61] introduces bridge building to add edges between distant nodes during insertion, improving search performance.

On the other hand, cluster-based methods [4, 6, 8, 11, 15, 21, 23, 28, 40, 51, 56] partition vectors using clustering algorithms such as k-means. Unlike graph-based methods that require updating multiple edges per operation, cluster-based approaches localize updates to individual partitions, reducing maintenance overhead under dynamic scenarios. Recent efforts [39, 40, 56] have focused on efficient updates in dynamic settings. SPFRESH [56] employs lightweight incremental rebalancing to handle index updates. Ada-IVF [39] selectively reindexes problematic partitions using temperature-based prioritization to minimize maintenance overhead. Quake [40] optimizes search under dynamic, skewed workloads through adaptive partition management driven by a cost model.

11 DISCUSSION

NAVIS primarily targets insertion because position seeking dominates concurrent search-update interference, while deletion is comparatively benign. OdinANN [20] reports that deletion under buffered merging interferes little with search, since deletion only removes a vertex from the graph and skips the position-seeking traversal that an insertion requires to find at least R adequate neighbors. Following OdinANN, deletion can be supported by enlarging the search candidate pool E_{search} , ignoring candidates invalidated in the indirection table, and triggering a bulk merge once the deleted-vector fraction crosses a threshold (e.g., 10%). Since NAVIS already maintains a host-memory indirection table for locality-driven decoupling’s out-of-place edge updates, this deletion path can be integrated naturally.

12 CONCLUSION

This paper presents NAVIS, an on-disk GVS system enabling concurrent, high-performance search and update with low position-seeking overhead. We identify position seeking as the main source of search-update interference, driven by unnecessary vector I/O in packed layouts and stale entrance graph. NAVIS mitigates these issues via selective vector reads, lightweight dynamic entrance graph maintenance, and an entrance graph-aware cache. Across benchmarks, NAVIS achieves up to 2.74 $\times$ insertion throughput. Even under such high insertion throughput, NAVIS provides up to 1.37 $\times$ higher search throughput and up to 25.26% lower search latency by substantially reducing search-update interference.

REFERENCES

- [1] Jens Axboe. 2019. Efficient IO with `io_uring`. https://kernel.dk/io_uring.pdf.
- [2] Artem Babenko and Victor S. Lempitsky. 2016. Efficient Indexing of Billion-Scale Datasets of Deep Descriptors. In *Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [3] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamee, Bhaskar Mitra, Tri Nguyen, Mir Rosenberg, Xia Song, Alina Stoica, Saurabh Tiwary, and Tong Wang. 2016. MS MARCO: A Human Generated Machine Reading Comprehension Dataset. *arXiv preprint arXiv:1611.09268* (2016).
- [4] Dmitry Baranchuk, Artem Babenko, and Yury Malkov. 2018. Revisiting the Inverted Indices for Billion-Scale Approximate Nearest Neighbors. In *Proceedings of the 15th European Conference on Computer Vision (ECCV)*.
- [5] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*.
- [6] Kangqi Chen, Rakesh Nadig, Manos Frouzakis, Nika Mansouri Ghiasi, Yu Liang, Haiyu Mao, Jisung Park, Mohammad Sadrosadati, and Onur Mutlu. 2025. REIS: A High-Performance and Energy-Efficient Retrieval System with In-Storage Processing. In *Proceedings of the 52nd International Symposium on Computer Architecture (ISCA)*.
- [7] Patrick Chen, Wei-Cheng Chang, Jyun-Yu Jiang, Hsiang-Fu Yu, Inderjit Dhillon, and Cho-Jui Hsieh. 2023. FINGER: Fast Inference for Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the ACM Web Conference 2023 (WWW)*.
- [8] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-Efficient Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the Advances in Neural Information Processing Systems 34 (NeurIPS)*.
- [9] Benjamin Coleman, Santiago Segarra, Alex Smola, and Anshumali Shrivastava. 2022. Graph Reordering for Cache-Efficient Near Neighbor Search. In *Proceedings of the Advances in Neural Information Processing Systems 35 (NeurIPS)*.
- [10] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys)*.
- [11] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2025. The Faiss Library. *IEEE Transactions on Big Data (BigData)* (2025).
- [12] Gil Einziger, Roy Friedman, and Ben Manes. 2017. TinyLFU: A Highly Efficient Cache Admission Policy. *ACM Transactions on Storage (TOS)* 13, 4 (2017).
- [13] Chantat Eksombatchai, Pranav Jindal, Jerry Zitao Liu, Yuchen Liu, Rahul Sharma, Charles Sugnet, Mark Ulrich, and Jure Leskovec. 2018. Pixie: A System for Recommending 3+ Billion Items to 200+ Million Users in Real-Time. In *Proceedings of the 2018 World Wide Web Conference (WWW)*.
- [14] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search with the Navigating Spreading-Out Graph. *Proceedings of the VLDB Endowment* 12, 5 (2019).
- [15] Tiezheng Ge, Kaiming He, Qifa Le, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 36, 4 (2014).
- [16] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatra, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference 2023 (WWW)*.
- [17] Google. [n.d.]. YouTube. <https://blog.youtube/press/>.
- [18] Fabian Groh, Lukas Ruppert, Patrick Wieschollek, and Hendrik P. A. Lensch. 2023. GGNN: Graph-Based GPU Nearest Neighbor Search. *IEEE Transactions on Big Data (BigData)* 9, 1 (2023).
- [19] Hao Guo and Youyou Lu. 2025. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [20] Hao Guo and Youyou Lu. 2026. OdinANN: Direct Insert for Consistently Stable Performance in Billion-Scale Graph-Based Vector Search. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST)*.
- [21] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*.
- [22] Jiale Han, Austin Cheung, Yubai Wei, Zheng Yu, Xusheng Wang, Bing Zhu, and Yi Yang. 2025. RAG Meets Temporal Graphs: Time-Sensitive Modeling and Retrieval for Evolving Knowledge. *arXiv preprint arXiv:2510.13590* (2025).
- [23] Han-Wen Hu, Wei-Chen Wang, Yuan-Hao Chang, Yung-Chun Lee, Bo-Rong Lin, Huai-Mu Wang, Yen-Po Lin, Yu-Ming Huang, Chong-Ying Lee, Tzu-Hsiang Su, Chih-Chang Hsieh, Chia-Ming Hu, Yi-Ting Lai, Chung-Kuang Chen, Han-Sung Chen, Hsiang-Pang Li, Tei-Wei Kuo, Meng-Fan Chang, Keh-Chung Wang, Chun-Hsiung Hung, and Chih-Yuan Lu. 2022. ICE: An Intelligent Cognition Engine with 3D NAND-based In-Memory Computing for Vector Similarity Search Acceleration. In *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*.
- [24] Jui-Ting Huang, Ashish Sharma, Shuying Sun, Li Xia, David Zhang, Philip Pronin, Janani Padmanabhan, Giuseppe Ottaviano, and Linjun Yang. 2020. Embedding-based Retrieval in Facebook Search. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*.
- [25] Zan Huang, Wingyan Chung, Thian-Huat Ong, and Hsinchun Chen. 2002. A Graph-based Recommender System for Digital Library. In *Proceedings of the 2nd ACM/IEEE-CS Joint Conference on Digital Libraries (JCDL)*.
- [26] Shikhar Jaiswal, Ravishankar Krishnaswamy, Ankit Garg, Harsha Vardhan Simhadri, and Sheshansh Agrawal. 2022. OOD-DiskANN: Efficient and Scalable Graph ANNS for Out-of-Distribution Queries. *arXiv preprint arXiv:2211.12850* (2022).
- [27] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2023. CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC)*.
- [28] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 33, 1 (2011).
- [29] Jinyoung Kim, Dayoon Ko, and Gunhee Kim. 2024. DynamicER: Resolving Emerging Mentions to Dynamic Entities for RAG. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [30] Ji-Hoon Kim, Yeo-Reum Park, Jaeyoung Do, Soo-Young Ji, and Joo-Young Kim. 2023. Accelerating Large-Scale Graph-Based Nearest Neighbor Search on a Computational Storage Platform. *IEEE Transactions on Computers (TC)* 72, 1 (2023).
- [31] Sukjin Kim, Seongyeon Park, Si Ung Noh, Junguk Hong, Taehee Kwon, Hunseong Lim, and Jinho Lee. 2025. PathWeaver: A High-Throughput Multi-GPU System for Graph-Based Approximate Nearest Neighbor Search. In *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC)*.
- [32] Stanford Vision Lab. [n.d.]. ImageNet. <https://www.image-net.org/index.php>.
- [33] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Proceedings of the Advances in Neural Information Processing Systems 33 (NeurIPS)*.
- [34] Jie Li, Haifeng Liu, Chuanghua Gui, Jianyu Chen, Zhenyuan Ni, Ning Wang, and Yuan Chen. 2018. The Design and Implementation of a Real Time Visual Search System on JD E-commerce Platform. In *Proceedings of the 19th International Middleware Conference Industry (Middleware)*.
- [35] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2020. Approximate Nearest Neighbor Search on High Dimensional Data – Experiments, Analyses, and Improvement. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 32, 8 (2020).
- [36] Jiahao Lou, Quan Yu, Shufeng Gong, Song Yu, Yanfeng Zhang, and Ge Yu. 2025. DGAI: Decoupled On-Disk Graph-Based ANN Index for Efficient Updates and Queries. *arXiv preprint arXiv:2510.25401* (2025).
- [37] Anton Lozhkov, Loubna Ben Allal, Leandro von Werra, and Thomas Wolf. 2024. FineWeb-Edu: the Finest Collection of Educational Content. <https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu>.
- [38] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 42, 4 (2020).
- [39] Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Umar Farooq Minhas, Jeffery Pound, Cedric Renggli, Nima Reyhani, Ihab F. Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. 2024. Incremental IVF Index Maintenance for Streaming Vector Search. *arXiv preprint arXiv:2411.00970* (2024).
- [40] Jason Mohoney, Devesh Sarda, Mengze Tang, Shihabur Rahman Chowdhury, Anil Pacaci, Ihab F. Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. 2025. Quake: Adaptive Indexing for Vector Search. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [41] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. 2024. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs. In *Proceedings of the 2024 IEEE 40th International Conference on Data Engineering (ICDE)*.
- [42] OpenAI. 2024. Embeddings – OpenAI API. <https://developers.openai.com/api/docs/guides/embeddings>.
- [43] OpenAI. 2024. New embedding models and API updates. <https://openai.com/index/new-embedding-models-and-api-updates/>.
- [44] Yu Pan, Jianxin Sun, and Hongfeng Yu. 2023. LM-DiskANN: Low Memory Footprint in Disk-Native Dynamic Graph-Based ANN Indexing. In *Proceedings*

- of the 2023 IEEE International Conference on Big Data (BigData).
- [45] Qdrant. 2025. Qdrant: High-Performance Vector Search at Scale. <https://qdrant.tech/>.
 - [46] Ziyue Qiu, Juncheng Yang, Juncheng Zhang, Cheng Li, Xiaosong Ma, Qi Chen, Mao Yang, and Yinlong Xu. 2023. FrozenHot Cache: Rethinking Cache Management for Modern Hardware. In *Proceedings of the 18th European Conference on Computer Systems (EuroSys)*.
 - [47] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*.
 - [48] Jie Ren, Minjia Zhang, and Dong Li. 2020. HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory. In *Proceedings of the Advances in Neural Information Processing Systems 33 (NeurIPS)*.
 - [49] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. *arXiv preprint arXiv:2105.09613* (2021).
 - [50] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node. In *Proceedings of the Advances in Neural Information Processing Systems 32 (NeurIPS)*.
 - [51] Philip Sun, David Simcha, Dave Dopson, Ruiqi Guo, and Sanjiv Kumar. 2023. SOAR: Improved Indexing for Approximate Nearest Neighbor Search. In *Proceedings of the Advances in Neural Information Processing Systems 36 (NeurIPS)*.
 - [52] Bing Tian, Haikun Liu, Zhuohui Duan, Xiaofei Liao, Hai Jin, and Yu Zhang. 2024. Scalable Billion-point Approximate Nearest Neighbor Search Using SmartSSDs. In *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC)*.
 - [53] Henrique Schechter Vera, Sahil Dua, Biao Zhang, Daniel Salz, Ryan Mullins, Sindhu Raghuram Panyam, Sara Smoot, Iftekhar Naim, Joe Zou, Feiyang Chen, Daniel Cer, Alice Lisak, Min Choi, Lucas Gonzalez, Omar Sanseviero, Glenn Cameron, Ian Ballantyne, Kat Black, Kaifeng Chen, Weiyi Wang, Zhe Li, Gus Martins, Jinhyuk Lee, Mark Sherwood, Juyeong Ji, Renjie Wu, Jingxiao Zheng, Jyotinder Singh, Abheesh Sharma, Divyashree Sreepathihalli, Aashi Jain, Adham Elarabawy, AJ Co, Andreas Doumanoglou, Babak Samari, Ben Hora, Brian Potetz, Dahun Kim, Enrique Alfonseca, Fedor Moiseev, Feng Han, Frank Palma Gomez, Gustavo Hernández Abrego, Hesen Zhang, Hui Hui, Jay Han, Karan Gill, Ke Chen, Koert Chen, Madhuri Shanbhogue, Michael Boratko, Paul Suganthan, Sai Meher Karthik Duddu, Sandeep Mariserla, Setareh Ariafer, Shanfeng Zhang, Shijie Zhang, Simon Baumgartner, Sonam Goenka, Steve Qiu, Tanmaya Dabral, Trevor Walker, Vikram Rao, Waleed Khawaja, Wenlei Zhou, Xiaoqi Ren, Ye Xia, Yichang Chen, Yi-Ting Chen, Zhe Dong, Zhongli Ding, Francesco Visin, Gaël Liu, Jiageng Zhang, Kathleen Kenealy, Michelle Casbon, Ravin Kumar, Thomas Mesnard, Zach Gleicher, Cormac Brick, Olivier Lacombe, Adam Roberts, Qin Yin, Yunhsuan Sung, Raphael Hoffmann, Tris Warkentin, Armand Joulin, Tom Duerig, and Mojtaba Seyedhosseini. 2025. EmbeddingGemma: Powerful and Lightweight Text Representations. *arXiv preprint arXiv:2509.20354* (2025).
 - [54] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proceedings of the ACM on Management of Data (PACMMOD)* 2, 1 (2024).
 - [55] Haike Xu, Magdalen Dobson Manohar, Philip A. Bernstein, Badrish Chandramouli, Richard Wen, and Harsha Vardhan Simhadri. 2025. In-Place Updates of a Graph Index for Streaming Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2502.13826* (2025).
 - [56] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, Peng Cheng, and Mao Yang. 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*.
 - [57] Song Yu, Shengyuan Lin, Shufeng Gong, Yongqing Xie, Ruicheng Liu, Yijie Zhou, Ji Sun, Yanfeng Zhang, Guoliang Li, and Ge Yu. 2025. A Topology-Aware Localized Update Strategy for Graph-Based ANN Index. *Proceedings of the VLDB Endowment* 19, 3 (2025).
 - [58] Yuanhang Yu, Dong Wen, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2022. GPU-accelerated Proximity Graph Approximate Nearest Neighbor Search and Construction. In *Proceedings of the 2022 IEEE 38th International Conference on Data Engineering (ICDE)*.
 - [59] Shulin Zeng, Zhenhua Zhu, Jun Liu, Haoyu Zhang, Guohao Dai, Zixuan Zhou, Shuangchen Li, Xuefei Ning, Yuan Xie, Huazhong Yang, and Yu Wang. 2023. DF-GAS: a Distributed FPGA-as-a-Service Architecture towards Billion-Scale Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the 56th IEEE/ACM International Symposium on Microarchitecture (MICRO)*.
 - [60] Minjia Zhang, Wenhan Wang, and Yuxiong He. 2022. GraSP: Optimizing Graph-based Nearest Neighbor Search with Subgraph Sampling and Pruning. In *Proceedings of the 15th ACM International Conference on Web Search and Data Mining (WSDM)*.
 - [61] Ziyu Zhang, Yuanhao Wei, Joshua Engels, and Julian Shun. 2025. CleANN: Efficient Full Dynamism in Graph-based Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2507.19802* (2025).
 - [62] Weijie Zhao, Shulong Tan, and Ping Li. 2020. SONG: Approximate Nearest Neighbor Search on GPU. In *Proceedings of the 2020 IEEE 36th International Conference on Data Engineering (ICDE)*.
 - [63] Lei Zhu, Chaoqun Zheng, Weili Guan, Jingjing Li, Yang Yang, and Heng Tao Shen. 2024. Multi-Modal Hashing for Efficient Multimedia Retrieval: A Survey. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 36, 1 (2024).
 - [64] Zhenhua Zhu, Jun Liu, Guohao Dai, Shulin Zeng, Bing Li, Huazhong Yang, and Yu Wang. 2023. Processing-In-Hierarchical-Memory Architecture for Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the 60th ACM/IEEE Design Automation Conference (DAC)*.